

言語処理系 第7回
コード生成・簡易コンパイラ

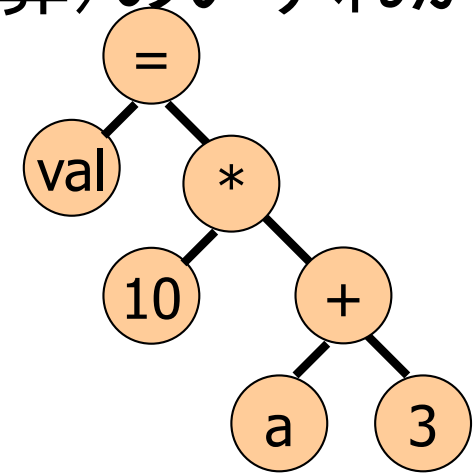
木村啓二
keiji@waseda.jp

第5回演習問題(1)解答例

- 式"(i+j)"を先の「LR(1)の状態遷移図（一部）」で読み取ることを考える．"i"は状態遷移によりEまで還元されるが、Eに還元されるのはどの状態か？
  - **J2′**
- さらに上記の状態からどの記号の先読みによりシフトあるいは還元が決まるか？
  - **+**

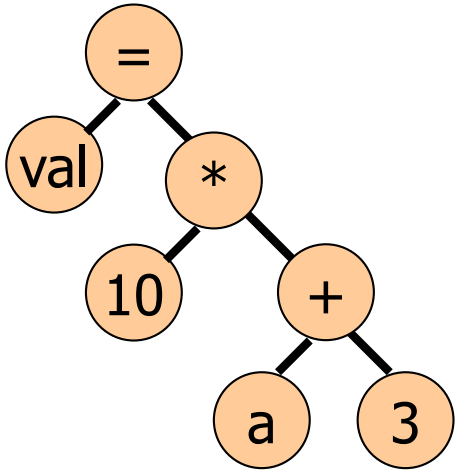
第5回演習問題(2)

- 以下の構文木に一時変数を割り当て、その後四つ組みを示せ。一時変数は先のスライドの通り深さ優先で番号を割り当てていくものとする。
- 四つ組みのフォーマットは(ope, dst, src1, src2)とする。ただし、dstは結果オペランド、src1, src2はそれぞれソースオペランドの1,2。また、opeはasgn（代入）, mul（乗算）, add（加算）のいずれか。

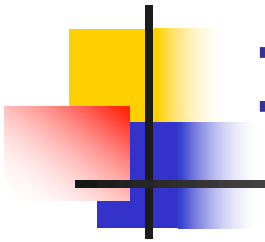


第5回演習問題(2)解答例

- 以下の構文木に一時変数を割り当て、その後四つ組みを示せ



(add,T1,a,3)
(mul,T2,10,T1)
(asgn,val,T2)



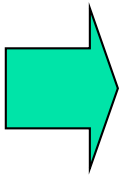
コード生成

- ソースプログラムの構文を中間表現へ
  - 前回は主に式の部分の変換
  - ソースプログラム→構文木→中間表現
  - 配列や構造体アクセスなど，アドレス計算を伴う低レベルな表現に変換される場合が多い
- 中間表現からターゲットアーキテクチャのアセンブリ命令を生成
  - 効率よい命令選択が求められる
- レジスタの利用

構文木から四つ組みに（特に文の扱い）

- 構文木を巡回しつつノードの種類に見合った四つ組み生成関数を呼び出す
- 条件分岐やswitch-caseは場合が多くなると効率悪いし読みにくい
  - 関数のポインタ(C)やクラスの継承(Javaなど)を使う

```
gen_stm(AST_Node *s)
{
  switch (s->op) {
  case  AST_ASSIGN:
    gen_assign(s); break;
  case  AST_IF:
    gen_if(s); break;
  case  AST_WHILE:
    gen_while(s); break;
  …
  }
}
```



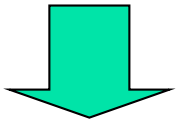
```
void (*gen_funcs[])() = {
    gen_none,
    …,
    gen_assign,
    gen_if,
    gen_while,
    …
};
gen_stm(AST_Node *s)
{
    gen_funcs[s->op](s);
}
```

今回の簡易コンパイラではこちら

条件分岐(if文)の構文木から四つ組みへの変換

get_label(): 新たなラベル番号を得る
gen_label(): ラベル文を生成する

if (E) S1 else S2



(jf E label1)
S1
(jmp label2)
(label1:)
S2
(label2:)

Eが偽なら
label1に飛ぶ

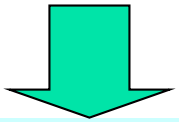
左記のコード列を
生成するgen_if()
関数を用意する

```
gen_if(AST_Node *s)
{
  条件式Eの生成
  l_else = get_label();
  l_end = get_label();
  l_elseを使ってjf文を生成
  S1の生成
  l_endを使ってjmp文を生成
  gen_label(l_else);
  S2の生成
  gen_label(l_end);
}
```

実際にはS2(else文)の
有無も考慮する必要有り

while文の構文木から四つ組みへの変換

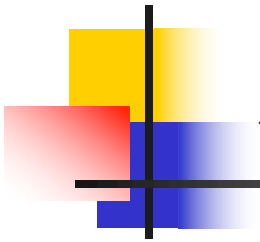
while (E) S



(label1:)
(jf E label2)
S
(jmp label1)
(label2:)

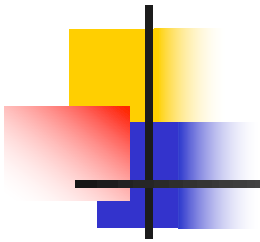
左記のコード列を
生成するgen_while()
関数を用意する

```
gen_while()
{
  l_begin = get_label();
  l_exit = get_label();
  gen_label(l_begin);
  条件式Eの生成
  l_exitを使ってjf文を生成
  Sの生成
  l_beginを使ってjmp文を生成
  gen_label(l_exit)
}
```



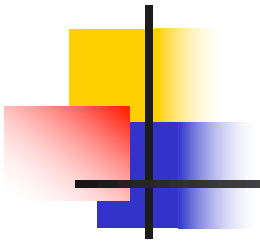
演習問題(1)

- 先のwhile文の変換に倣ってdo-while文の四組列を書け
  - do S while (E);が与えられるものとする。
- 同じくdo-while文のコード生成関数gen_dowhile()を書け
- Moodleでは選択問題



目的語の生成

- 中間語からターゲットCPUの目的語（機械語）を生成する
- 実行時間の短い命令を選択する
  - 最近はあまり選択の余地がない
- レジスタとメモリ間の転送命令を生成する

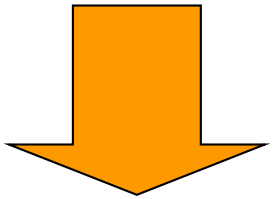


レジスタ

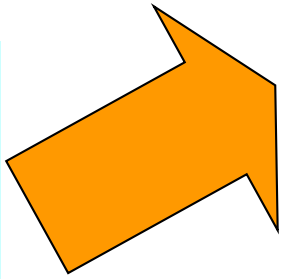
- なるべくメモリアクセスしないようなコード生成が重要
  - レジスタを有効利用する
  - とりあえず、使えるだけのレジスタを使い、足りなくなったらスタックに退避する、といった方法でもなんとかなる

目的語生成の(簡単な)例

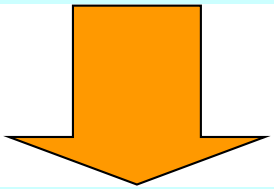
```
y=x+1;
```



```
# only body
  (
    (add,t1,x,1)
    (asgn,y,t1)
)
```



```
# x: x29-8
# y: x29-4
  (
    (load,t2,x)
    (loadi,t3,1)
    (add,t1,t2,t3)
    (asgn,y,t1)
    (store, y)
)
```

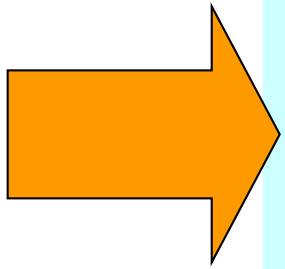


```
; only body
    ldr     w8,[x29-8]
    mov     w9,1
    add     w8,w8,w9
    str     w8,[x29-4]
```

目的語生成の例

```
if (a>0) val=10*(a+3);
else     val=10*a+3;
```

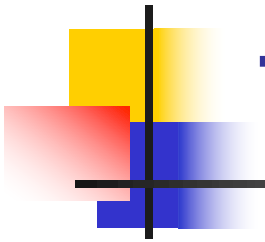
```
# body部分のみ
 (
  (ge,t1,a,0)
  (jne,t1,L1)
  (add,t2,a,3)
  (mul,t3,10,t2)
  (asgn,val,t3)
  (jmp,L2)
  (label,L1)
  (mul,t4,10,a)
  (add,t5,t4,3)
  (asgn,val,t5)
  (label,L2)
)
```



```
; body部分のみ
    ldr     w8,[x29-8]
    mov     w9,0
    cmp     w8,w9
    b.le    .L1
    ldr     w8,[x29,-8]
    mov     w9,3
    add     w8,w8,w9
    mov     w9,10
    mul     w9,w9,w8
    str     w9,[x29,-4]
    b      .L0
.L1:
    mov     w8,10
    ldr     w9,[x29,-8]
    mul     w8,w8,w9
    mov     w9,3
    add     w8,w8,w9
    str     w8,[x29,-4]
.L0:
```

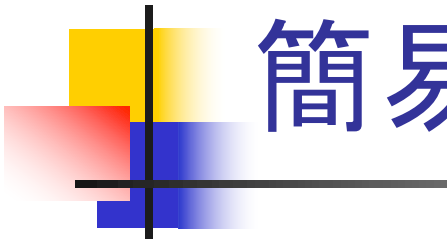
実習で扱う簡易コンパイラ: 簡易言語(tl)

- Cのサブセット
  - 変数はint型の変数のみ宣言できる
    - **配列も構造体も無し**
  - 定数は整定数のみ利用できる
  - 式は+,-,*,/,<,<=,>=,>,==,!=のみ利用できる
  - 文はif,while,do-while,forのみ利用できる
  - 整数値表示の用のput_int組み込み関数を利用可能



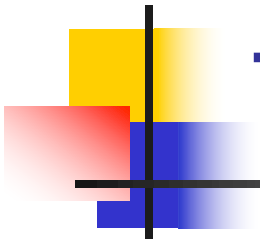
サンプル

```
main()
{
    int a, s;
    a = 1;
    s = 0;
    while (a <= 10) {
        s = s + a;
        a = a+1;
    }
    put_int(a);
    put_int(s);
}
```



簡易コンパイラ(tlc)

- tlのコンパイラ
- ASTから直接ARM(**64ビット**)コードを出力する
  - x64のコードも生成可能
- ビルド方法
  - MoodleよりTlc64.zipをダウンロード
  - Raspberry Pi4に転送
  - unzip tlc64.zip
    - doc/以下にドキュメントがあるのでPC側で参照すること
  - cd src
  - MakefileのPLATFORM=の行が"PLATFORM=RASPI"となっていること(先頭に"#"が無いこと)を確認する
  - make
    - tlcが生成される



tlcのテスト

- src/testにテスト用のサンプルがあるので作業ディレクトリをそこに移す
- ../tlc test1.c
- gcc –o test1 test1.s
- ./test1
  - 6が出力されたらOK

tlcが画面に出力するもの

- シンボルテーブルとASTの情報
- 例：test1.cの出力

```
SymTab
id(1)
 a #1, offset(-4)
 b #2, offset(-8)
root(main)
 func[ identifier(r0)(main)] ()
  l(3): declaration( identifier(r0)(a))
  l(4): declaration( identifier(r0)(b))
  l(5): stm_asign( exp_asign(r0)( identifier(r0)(b) const_int(r1)(5)))
  l(6): stm_asign( exp_asign(r1)( identifier(r1)(a) add(r0)( identifier(r0)(b)
const_int(r1)(1))))
  l(7): put_int( identifier(r0)(a))
```

シンボルテーブルの情報

変数aはスタックフレーム上のオフセット-4の位置に配置

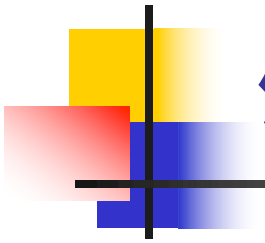
ここからASTの情報

変数aの宣言

代入文 b=5

代入文 a=b+1

l(整数)は行番号



処理の流れ(main.c)

```
main()
{
    // 略
    yyparse();  // 構文解析器呼び出し(字句解析は構文解析器から呼び出される)

    assign_memory(); // 変数配置決定
    assign_regs();  // レジスタ割り付け

    dump_symtab(); //シンボルテーブル情報を出力
    dump_ast();  // ASTの出力

    gen_code(); // コード生成

}
```

構文解析木の定義: tl_gram.y

繰り返し文（ループ）の定義

while文の定義

3番目

5番目

```
iteration_statement
        : TOKEN_WHILE TOKEN_LPAREN expression TOKEN_RPAREN statement
        { $$ = act_while_stm($3, $5); }
```

while文のASTを作成する関数
parse_action.cで定義

構文木の構築: parse_action.c

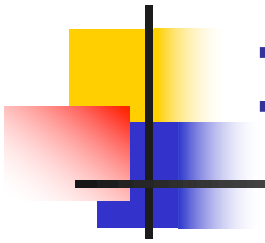
継続条件式のポインタ

ループ本体（文）のポインタ

```
AST_Node*
act_while_stm(AST_Node *e, AST_Node *s)
{
    AST_Node *ret = create_AST_Stm(AST_STM_WHILE, yylineno);
    ret->child[0] = e;
    ret->child[1] = s;
    if (e != NULL) {
        e->parent = ret;
    }
    if (s != NULL) {
        s->parent = ret;
    }
    return ret;
}
```

文ノードの構築

e->kindはAST_KIND_EXP
s->kindはAST_KIND_STM
であることを期待している



コード生成：cg.c, arch_arm64.c

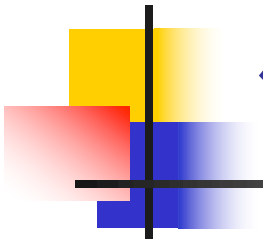
- 基本構造は先のコード生成のスライドの通り
- 実際のコード生成の前にレジスタ割り当てを行う
- Raspberry Pi4のARM依存部分はarch_arm64.cに実装されている
  - （参考）x64依存部分はarch_x64.cにある

課題（締切は6月18日深夜）

- 簡易コンパイラtlcに関する以下のスライド以降の作業を行いレポートとしてまとめよ（10点満点）
  - 作業1, 2, 3で基準点7点、作業4はオプション
- 表紙をつけること
  - 学籍番号・氏名・課題出題日・課題提出日を明記する
- 作業の手順を文章で明記すること
- 作業1,2の実行結果ならびに作業3の調査結果を示すこと
- 各実行結果・調査結果に対して考察を行うこと

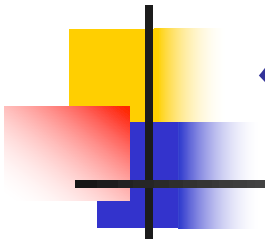
レポートに関する注意（再掲）

- 他の文献から記述やデータを引用してあった場合、**引用もとを明記しないレポートは0点**
- レポートは必ず自分で仕上げる
  - **類似レポートはどれがオリジナルであっても全て0点**
  - **「てにをは」や単語の順序を変えた程度のものも類似レポートと見なす**



作業1

- 学籍番号に応じて以下の文を用いた簡単なプログラムを書き、tlcが出力したログを報告せよ
  - 学籍番号を3で割った余りが1
    - while文
  - 学籍番号を3で割った余りが2
    - for文
  - 学籍番号を3で割った余りが0
    - if-else文
- 行番号に着目して考察すること



作業2

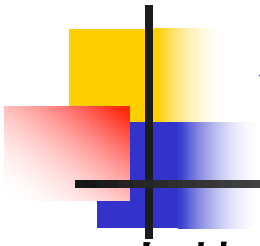
- do-while文に関して以下の作業（赤字部分）を行え
  - 文法
    - **do** *statement* **while (** *expression* **) ;**
  - do-whileのプログラム例：

```
do {
  a = a+1;
} while (a < 10);
```

  - while文との文法的、及び動作的な違いを簡潔に説明せよ
  - 以下のwhileループのあるプログラムからtlcが生成したアセンブリを参考に、上記do-whileのプログラム例からどのようなアセンブリを生成すべきか答えよ

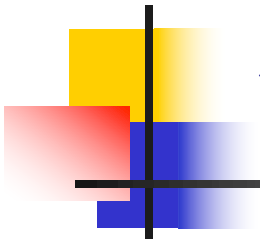
```
while (a < 10) {
    a = a+1;
}
```

  - tlcにdo-whileを実装せよ



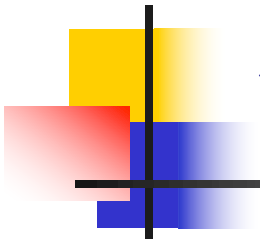
作業2（ヒント）

- 実装作業箇所（以下のコード中に“REPORT3”で作業箇所がマークしてある）
  - tl_gram.y, parse_action.h, parse_action.c, ast_c, cg.c
- while文, for文の実装を参考にすると良い
  - 継続条件の評価コード生成部分に注意する
  - 具体的にはcg.cのgen_stm_rel()とgen_exp_rel()
- **ソースコードを修正したらmakeを実行してtlcを再ビルドする必要がある**
- テストコードを用いてテストすること
- tlcの修正部分、出力ログ、生成されたアセンブリコード、テストコードの実行結果を報告する
- 参考にするコードがある以上、誰が実装してもそれほどかわらないはずなので、実装したコードやテストコードの類似性は問わない
- **考察は必ず各自で書くこと（類似性を厳しくチェックする）**



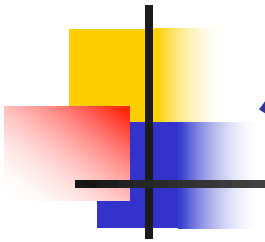
作業3

- 関数sumを呼び出すテストプログラムtest/test22.cをコンパイルし、生成されたアセンブリコードを確認して以下に答えよ:
  - 関数sumの中で二つの引数が、x29の指すアドレスを起点としてどの位置に格納されているか簡単に図示せよ
  - 第8回の資料をMoodleにアップロードするので、そちらの説明も参考にしてください



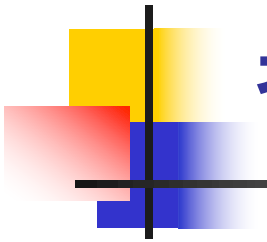
作業4(オプション)

- tlcを拡張せよ
- 例
  - 配列を使えるようにする
  - ポインタを使えるようにする
  - 割り算を使えるようにする
  - なんらかの最適化を行うようにする



バグについて

- あると思うので適宜対応してください
  - 気がついたら教えてください



参考：Makefile

- makeの動作を記述する
  - どのファイルがどのように生成（コンパイル）されるかなど

```
ast.o : ast.c ast.h util.h
（ast.oはast.c, ast.h, util.hから生成（コンパイル）される）
（→ast.c, ast.h, util.hが修正されたらast.oは再生成される）
```

```
.c.o:
        gcc $(CFLAGS) $(TARGET_FLAG) -c $<
（～.c（$<が対応）を～.oにコンパイルする）
```

```
.l.c:
        flex -o $@ $<
（～.l（$<が対応）からflexにより～.c（$@に対応）を生成する）
```

参考：利用するARM64命令
レジスタ

- レジスタ
  - 汎用レジスタ
    - x0からx30の64ビットレジスタ
    - x30はbl命令で関数呼び出しを行ったときの戻りアドレスが格納される（リンクレジスタ）
    - 32ビットレジスタとして使うときはw0からw30
    - tlcではw8,w9,w10の3つを演算用に使う
  - スタックポインタ、フレームポインタ（次回）
    - スタックポインタ: sp
    - フレームポインタ: x29

参考：利用するARM64命令 mov, ldr, str命令

- レジスタは<reg1>等として表す
- mov
  - mov <reg1>, <reg2>
    - <reg2>の値を<reg1>にコピーする
  - mov <reg1>,<定数>
    - <定数>（16ビット定数）を<reg1>にコピーする
    - <定数>はラベルでもOK
- ldr
  - ldr <reg1>,[<reg2>,<定数>]
    - <reg2>+<定数>のアドレスの値を<reg1>にロードする
- str
  - str <reg1>,[<reg2>,<定数>]
    - <reg1>の値を<reg2>+<定数>のアドレスにストアする

参考：利用するARM64命令
演算命令

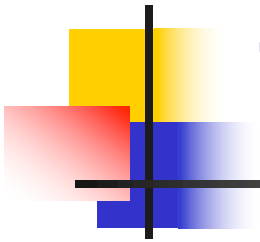
- neg <reg1>,<reg2>
  - -<reg2>の値を<reg1>に入れる
- add, sub, mul
  - 基本形
    - ope <reg1>, <reg2>, <reg3>
      - <reg2>ope<reg3>の値を<reg1>に入れる
    - ope <reg1>,<reg2>,<定数>
      - <reg2>ope<定数>の値を<reg1>に入れる
      - <定数>は16ビット定数

参考：利用するARM64命令 比較命令・分岐命令

- cmp <reg1>,<reg2>
  - <reg1>と<reg2>の比較してフラグを操作する
- cset <reg1>, CC
  - CCにはlt, gt, le, ge, eq, neのいずれかが入る
  - 比較結果（フラグの値）が成立していれば<reg1>に1を、成立していなければ0をセットする
- b.ge, b.le, b.gt, b.lt, b.ne, b.eq
  - 基本形
    - b.CC <ラベル>
      - 比較結果（フラグの値）によって<ラベル>に分岐する
- b <ラベル>
  - <ラベル>にジャンプする

参考：利用するARM64命令
関数呼び出し関連命令

- stp x29,x30, [sp, <定数>]!
  - sp+<定数>を新たなspの値とする
  - 更新されたspの指すアドレスにx29とx30を格納する
  - スタックにx29,x30をpushするのと同じ
- ldp x29,x30,[sp],<定数>
  - spの指すアドレスからx29とx30を読み出し、spの値をsp+<定数>に更新する
  - スタックからx29,x30をpopするのと同じ
- bl <ラベル>
  - bl命令の次の命令のアドレス（PC+4）をx30に格納し、<ラベル>にジャンプする
- ret
  - x30に格納されているアドレスにジャンプ（リターン）する（関数から戻る）



まとめ

- コード生成
  - 中間表現に従ってコード生成する
  - 関数呼び出しは次回
  - レジスタ割り付けは別の回で
- 簡易コンパイラ
  - 色々拡張してみてください